

Experiment - 2

Develop a mapreduce program to implement Matrix Multiplication

Remove the existing eclipse IDE in the system.

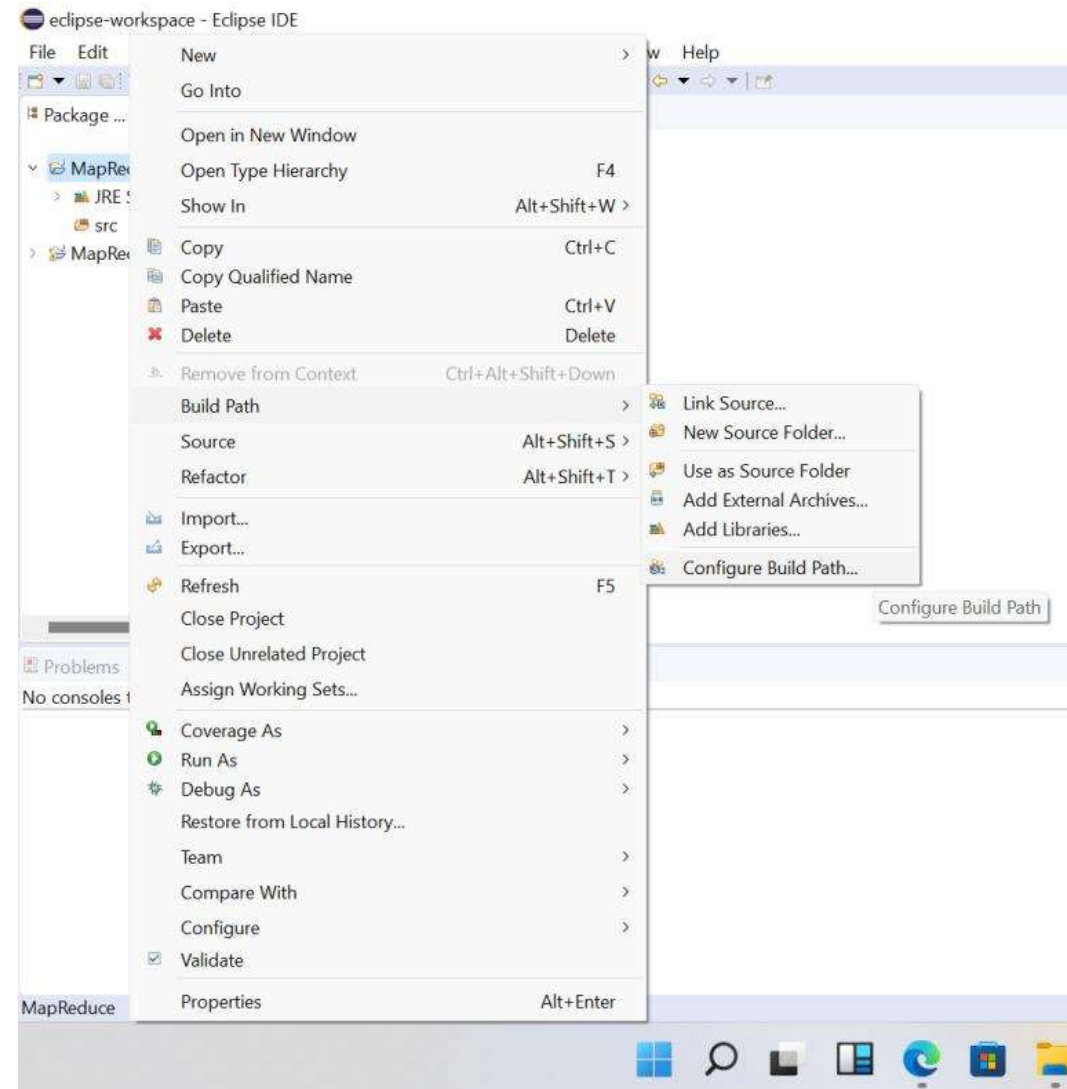
Refer a below link to download and install Eclipse version: Eclipse IDE 2019-12

<https://www.eclipse.org/downloads/packages/release/2019-12/r/eclipse-ide-java-developers>

Step 1: Open Eclipse, create a Java project by any name, create a package and give a name for it.

Add external jars which are required for the programs by following the steps given below:

- 1) Right click on the project that have been created, choose an option called as “build path”, then choose “configuration build path”.
- 2) choose “Java Build Path”, then “Libraries” and then “Add External JAR’s”. Refer to the image in the next slide.



Properties for MapReduce

- > Resource
- Builders
- Coverage
- Java Build Path
- > Java Code Style
- > Java Compiler
- > Java Editor
- Javadoc Location
- Project Natures
- Project Reference:
- Run/Debug Settir
- > Task Repository
- Task Tags
- > Validation
- WikiText

Java Build Path

Source Projects Libraries Order and Export Module Dependencies

JARs and class folders on the build path:

> JRE System Library [JavaSE-1.8]

Add JARs...

Add External JARs...

Add Variable...

Add Library...

Add Class Folder...

Add External Class Folder...

Edit...

Remove

Migrate JAR File...

Apply



Apply and Close

Cancel

3) After choosing an option “Add External JAR’s”, select all the JAR files in the path **C:\hadoop\share\hadoop**

Select each folder of it, and choose all the JAR files and add it to the JRE System library.

4) Create 3 class files, one for Mapper, Reducer and Driver.

In our program, we are going to create files named:

- MatrixMultiplication.java
- MatrixMultiplicationMapper.java
- MatrixMultiplicationReducer.java

Add the code in all these files, and press Ctrl+S to save it.

5) Create a jar file of your project, by right clicking on created java project, then choose “export”, in that, choose “Java”, and then choose “JAR file” option given, then click on “Next”. Give a JAR file name that you want to create and give a path where it need to be saved. Refer the screenshots given in the next slide.

eclipse-workspace - MapReduce/src/com/mapreduce/matrix/MatrixMultip

File Edit Source Refactor Navigate Search Project Run Wind

The screenshot shows the Eclipse IDE's context menu for a package. The menu items are as follows:

- New
- Go Into
- Open in New Window
- Open Type Hierarchy (F4)
- Show In (Alt+Shift+W)
- Copy (Ctrl+C)
- Copy Qualified Name
- Paste (Ctrl+V)
- Delete (Delete)
- Remove from Context (Ctrl+Alt+Shift+Down)
- Build Path
- Source (Alt+Shift+S)
- Refactor (Alt+Shift+T)
- Import...
- Export... (highlighted, with a tooltip showing 'Export')
- Refresh (F5)
- Close Project
- Assign Working Sets...
- Coverage As
- Run As
- Debug As

Export

Select

Export resources into a JAR file on the local file system.

Select an export wizard:

type filter text

- Java
 - JAR file
 - Javadoc
 - Runnable JAR file
- Run/Debug



< Back

Next >

Finish

JAR Export

JAR File Specification



Define which resources should be exported into the JAR.

Select the resources to export:

>  MapReduce

☒  .classpath

☒  .project

☒ Export generated class files and resources

☐ Export all output folders for checked projects

☐ Export Java source files and resources

☐ Export refactorings for checked projects. [Select refactorings...](#)

Select the export destination:

JAR file:

E:\Matrixmul.jar

▼

Browse...

Options:

☒ Compress the contents of the JAR file

☐ Add directory entries

☐ Overwrite existing files without warning



< Back

Next >

Finish

Cancel

After creating the JAR file of the project, follow the below given commands.

Step 2: Open **command prompt** in **administrator mode**.

Step 3: start all the daemons of hadoop by giving a command: **start-all.cmd**

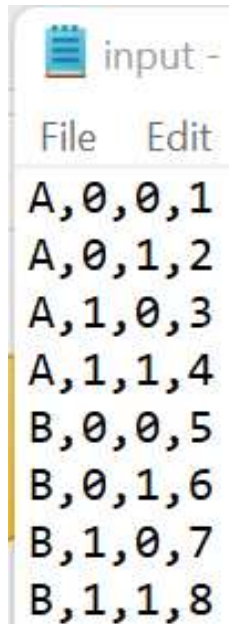
```
C:\Windows\system32>cd \  
  
C:\>cd hadoop  
  
C:\hadoop>cd sbin  
  
C:\hadoop\sbin>start-all.cmd
```

Step 4: create a directory in hdfs by using a command:

hadoop fs -mkdir /directoryname

```
C:\hadoop\sbin>hadoop fs -mkdir /input_matrix
```

Step 5: create a text file in any of the directory in your computer. Store the inputs to be given for the program. Text file name should be : **input.txt**



input.txt

File	Edit
A,0,0,1	
A,0,1,2	
A,1,0,3	
A,1,1,4	
B,0,0,5	
B,0,1,6	
B,1,0,7	
B,1,1,8	

Step 6: add the text file to the directory that is created in HDFS previously by using the given command: `C:\hadoop\sbin>hadoop fs -put E:\input.txt /input_matrix`

To verify whether the file exists or not in the directory of hdfs, check in the localhost:9870

Or

By typing a command: `C:\hadoop\sbin>hadoop fs -ls /input_matrix`

To display the contents of the text file in the command prompt, use a command as given below:

```
C:\hadoop\sbin>hadoop dfs -cat /input_matrix/input.txt
DEPRECATED: Use of this script to execute hdfs command is deprecated.
Instead use the hdfs command for it.
A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,4
B,0,0,5
B,0,1,6
B,1,0,7
B,1,1,8
```

Step 7: To execute the program, follow the below given command:

```
C:\hadoop\sbin>hadoop jar E:\Matrixmul.jar com.mapreduce.matrix/MatrixMultiplication /input_matrix/input.txt /output_matrix
```

Path of projects JAR file

Path(path from the package of current working java project) of the java file which consists of main method.

It's a HDFS path where the input.txt file is present, which is the input of the program.

It's a path where the output of the program will be stored in the form of file.

```
2025-03-15 22:32:55,810 INFO mapreduce.Job: map 0% reduce 0%
2025-03-15 22:33:02,986 INFO mapreduce.Job: map 100% reduce 0%
2025-03-15 22:33:11,096 INFO mapreduce.Job: map 100% reduce 100%
2025-03-15 22:33:11,106 INFO mapreduce.Job: Job job_1742057507536_0001 completed successfully
```

Step 8: To check the output, type a command as given below:

```
C:\hadoop\sbin>hadoop dfs -cat /output_matrix/*
```

```
0,0      19  
0,1      22  
1,0      43  
1,1      50
```

output

```
package com.mapreduce.matrix;

import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;

public class MatrixMultiplicationMapper extends Mapper<LongWritable, Text, Text, Text> {
    private static final int MATRIX_SIZE = 2; // Set to correct matrix dimensions (2x2 in your case)

    public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
        String[] tokens = value.toString().split(",");
        String matrixName = tokens[0]; // "A" or "B"
        int i = Integer.parseInt(tokens[1]);
        int j = Integer.parseInt(tokens[2]);
        int val = Integer.parseInt(tokens[3]);

        if (matrixName.equals("A")) {
            for (int k = 0; k < MATRIX_SIZE; k++) { // Iterate correctly
                context.write(new Text(i + "," + k), new Text("A," + j + "," + val));
            }
        } else if (matrixName.equals("B")) {
            for (int k = 0; k < MATRIX_SIZE; k++) { // Iterate correctly
                context.write(new Text(k + "," + j), new Text("B," + i + "," + val));
            }
        }
    }
}
```

import org.apache.hadoop.io.*;

- imports all classes inside the org.apache.hadoop.io package. These classes handle **Hadoop-specific data types** that are optimized for distributed computing.

import org.apache.hadoop.mapreduce.Mapper;

- This imports the **Mapper** class, which is a part of the Hadoop MapReduce framework.

import java.io.IOException;

- This imports the **IOException** class from Java's built-in **java.io** package.

```
package com.mapreduce.matrix;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;
import java.util.HashMap;

public class MatrixMultiplicationReducer extends Reducer<Text, Text, Text, IntWritable> {
    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
        HashMap<Integer, Integer> aValues = new HashMap<>();
        HashMap<Integer, Integer> bValues = new HashMap<>();

        for (Text val : values) {
            String[] parts = val.toString().split(",");
            if (parts[0].equals("A")) {
                aValues.put(Integer.parseInt(parts[1]), Integer.parseInt(parts[2]));
            } else {
                bValues.put(Integer.parseInt(parts[1]), Integer.parseInt(parts[2]));
            }
        }

        int sum = 0;
        for (int k : aValues.keySet()) {
            if (bValues.containsKey(k)) {
                sum += aValues.get(k) * bValues.get(k);
            }
        }

        context.write(key, new IntWritable(sum));
    }
}
```

<code>org.apache.hadoop.io.*</code>	Brings in Hadoop's optimized data types (<code>Text</code> , <code>IntWritable</code>).
<code>org.apache.hadoop.mapreduce.Reducer</code>	Imports the Hadoop Reducer class to define the reduce function.
<code>java.io.IOException</code>	Handles input-output exceptions during file processing.
<code>java.util.HashMap</code>	Provides a data structure to store intermediate values (A and B matrix elements).

```
package com.mapreduce.matrix;
import org.apache.hadoop.conf.Configuration;

public class MatrixMultiplication {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Matrix Multiplication");

        job.setJarByClass(MatrixMultiplication.class);
        job.setMapperClass(MatrixMultiplicationMapper.class);
        job.setReducerClass(MatrixMultiplicationReducer.class);

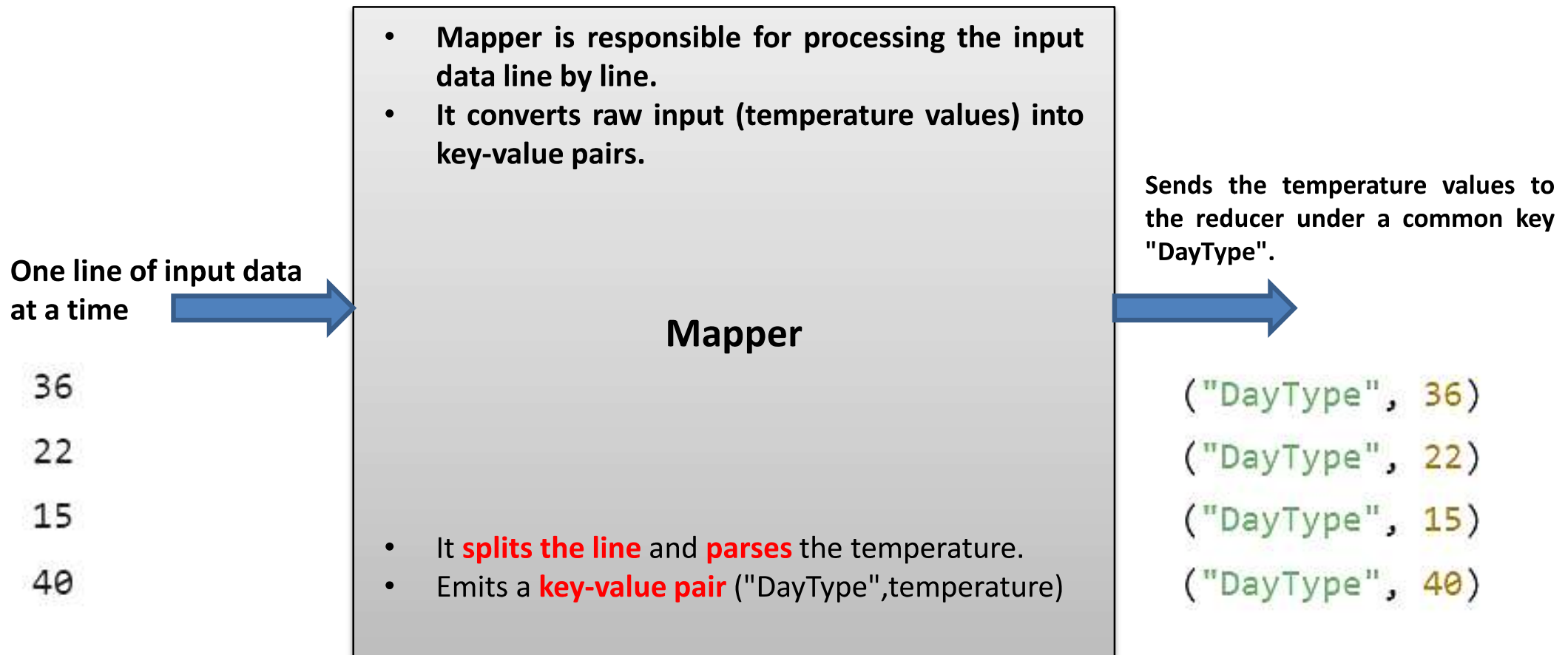
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Exp - 3

Develop a Map Reduce program that mines weather data and displays appropriate messages indicating the weather conditions of the day.



Receives a key
("DayType") and a list
of temperature values.

("DayType", [36, 22, 15, 40])

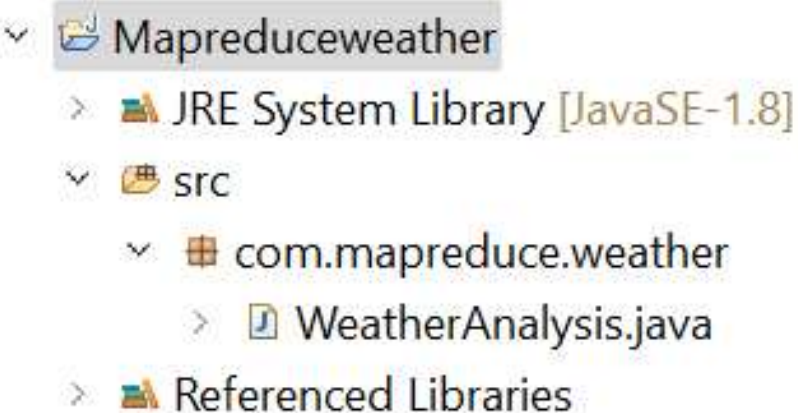
- The **Reducer** processes the key-value pairs received from the **Mapper**.
- It categorizes each temperature based on conditions:
 - > 35°C → Hot Day
 - 20–35°C → Pleasant Day
 - < 20°C → Cold Day

Reducer

- Iterates through each temperature.
- Classifies the type of day based on temperature.

Emits (**temperature, condition**)
pairs.

36C	Hot Day
22C	Pleasant Day
15C	Cold Day
40C	Hot Day



36
22
15
40



```
C:\Windows\system32>cd \  
  
C:\>cd hadoop  
  
C:\hadoop>cd sbin
```

```
C:\hadoop\sbin>start-all.cmd
```

```
C:\hadoop\sbin>jps  
3552 DataNode  
4388 ResourceManager  
6744  
8744 NodeManager  
12188 Jps  
18412 NameNode
```

```
C:\hadoop\sbin>hadoop fs -mkdir /input_fold  
  
C:\hadoop\sbin>hadoop fs -put E:\input.txt /input_fold  
  
C:\hadoop\sbin>hadoop dfs -cat /input_fold/input.txt  
DEPRECATED: Use of this script to execute hdfs command is deprecated.  
Instead use the hdfs command for it.  
36  
22  
15  
40
```

```
C:\hadoop\sbin>hadoop jar E:\Weathermr.jar com.mapreduce.weather/WeatherAnalysis /input_fold/input.txt /output_fold
```

```
map 0% reduce 0%  
map 100% reduce 0%  
map 100% reduce 100%
```

- Add all the external JAR's from the path C:\hadoop\share\hadoop

```
C:\hadoop\sbin>hadoop dfs -cat /output_fold/*
```

```
DEPRECATED: Use of this script to execute hdfs command is deprecated.  
Instead use the hdfs command for it.
```

```
40C      Hot Day
```

```
15C      Cold Day
```

```
22C      Pleasant Day
```

```
36C      Hot Day
```

```
package com.mapreduce.weather;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;
public class WeatherAnalysis
{
    public static class WeatherMapper extends Mapper<Object, Text, Text, IntWritable>
    {
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException
        {
            String[] fields = value.toString().split(",");
            if (fields.length == 1)
            {
                int temperature = Integer.parseInt(fields[0]);
                context.write(new Text("DayType"), new IntWritable(temperature));
            }
        }
    }
}
```

```
public static class WeatherReducer extends Reducer<Text, IntWritable, Text, Text>
{
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException, InterruptedException
    {
        for (IntWritable val : values)
        {
            int temp = val.get();
            String condition;

            if (temp > 35)
            {
                condition = "Hot Day";
            }
            else if (temp >= 20 && temp <= 35)
            {
                condition = "Pleasant Day";
            }
            else
            {
                condition = "Cold Day";
            }
            context.write(new Text(temp + "C"), new Text(condition));
        }
    }
}
```

```
public static void main(String[] args) throws Exception
{
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Weather Analysis");

    job.setJarByClass(WeatherAnalysis.class);
    job.setMapperClass(WeatherMapper.class);
    job.setReducerClass(WeatherReducer.class);

    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(IntWritable.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

Experiment 4

Develop a MapReduce program to find the tags associated with each movie by analyzing movie lens data.

Experiment 4

Develop a MapReduce program to find the tags associated with each movie by analyzing movie lens data.

Field 1: Record ID (Ignored)

Field 2: Movie ID (Important)

Field 3: Tag (Important)

Field 4: Timestamp (Ignored)

1,100,Action,1234567890
2,100,Thriller,1234567891
3,101,Romance,1234567892
4,102,Drama,1234567893
5,100,Adventure,1234567894

One line of input data
at a time

- Mapper is responsible for processing the input data line by line.

Mapper

- Each line is **read and converted** to a string
- The line is **split by commas**.
- The **Movie ID** and **Tag** are **extracted**
- The key-value pair (**Movie ID, Tag**) is **written to context**

(100, Action)
(100, Thriller)
(101, Romance)
(102, Drama)
(100, Adventure)

**Shuffle and sort
phase** (which
will be
happened
internally just
after the
mapper phase
by the hadoop)

100	[Action, Thriller, Adventure]
101	[Romance]
102	[Drama]

- The **Reducer** in Hadoop MapReduce is responsible for **aggregating and summarizing** the intermediate output generated by the **Mapper**.
- Its primary purpose is to **process grouped key-value pairs** after the shuffle and sort phase and produce the **final output**.



```
Key: 100  Values: [Action, Thriller, Adventure]
Key: 101  Values: [Romance]
Key: 102  Values: [Drama]
```

Reducer

- For each key (Movie ID), it loops through all associated tags.
- It uses a **StringBuilder** to **concatenate the tags**.
- Tags are separated by a **comma and space (", ")**.
- Once all tags for a particular Movie ID are concatenated, the result is written to the context



```
100  Action, Thriller, Adventure
101  Romance
102  Drama
```

- ▼ Movietags
 - > JRE System Library [JavaSE-1.8]
 - ▼ src
 - ▼ com.mapreduce.movie
 - > MovieTagMapperReducer.java
 - > Referenced Libraries

```
C:\Windows\system32>cd \
C:\>cd hadoop
C:\hadoop>cd sbin
C:\hadoop\sbin>start-all.cmd
```

```
C:\hadoop\sbin>jps
18800 Jps
7664 NameNode
7856 ResourceManager
8384
11768 DataNode
6200 NodeManager
```

movieinput - Notepad

	File	Edit	Format	View	Help
1,100,	Action,	1234567890			
2,100,	Thriller,	1234567891			
3,101,	Romance,	1234567892			
4,102,	Drama,	1234567893			
5,100,	Adventure,	1234567894			

```
C:\hadoop\sbin>hadoop fs -mkdir /input_doc
C:\hadoop\sbin>hadoop fs -put E:\movieinput.txt /input_doc
C:\hadoop\sbin>hadoop dfs -cat /input_doc/movieinput.txt
DEPRECATED: Use of this script to execute hdfs command is deprecated.
Instead use the hdfs command for it.
1,100,Action,1234567890
2,100,Thriller,1234567891
3,101,Romance,1234567892
4,102,Drama,1234567893
5,100,Adventure,1234567894
C:\hadoop\sbin>hadoop jar E:\Movie.jar com.mapreduce.movie/MovieTagMapperReducer /input_doc/movieinput.txt /output_doc
```

```
map 0% reduce 0%
map 100% reduce 0%
map 100% reduce 100%
```

```
C:\hadoop\sbin>hadoop dfs -cat /output_doc/*
DEPRECATED: Use of this script to execute hdfs command is deprecated.
Instead use the hdfs command for it.
100      Adventure, Thriller, Action
101      Romance
102      Drama
```

```
package com.mapreduce.movie;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;
public class MovieTagMapperReducer
{
    // Mapper Class
    public static class TagMapper extends Mapper<Object, Text, Text, Text>
    {
        public void map(Object key, Text value, Context context) throws IOException, InterruptedException
        {
            String line = value.toString();
            String[] fields = line.split(",");
            if (fields.length >= 4)
            { // Ensure the line has all required fields
                String movieId = fields[1]; // Movie ID is the second field
                String tag = fields[2]; // Tag is the third field
                context.write(new Text(movieId), new Text(tag));
            }
        }
    }
}
```

```
// Reducer Class
public static class TagReducer extends Reducer<Text, Text, Text, Text>
{
    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException
    {
        StringBuilder tags = new StringBuilder();
        for (Text val : values)
        {
            if (tags.length() > 0)
            {
                tags.append(", "); // Separate tags by comma
            }
            tags.append(val.toString());
        }
        context.write(key, new Text(tags.toString()));
    }
}
```

```
// Main Method
```

```
public static void main(String[] args) throws Exception
```

```
{
```

```
    Configuration conf = new Configuration();
```

```
    Job job = Job.getInstance(conf, "Movie Tags");
```

```
    job.setJarByClass(MovieTagMapperReducer.class);
```

```
    job.setMapperClass(TagMapper.class);
```

```
    job.setReducerClass(TagReducer.class);
```

```
    // Explicitly setting Mapper output key and value classes
```

```
    job.setMapOutputKeyClass(Text.class);
```

```
    job.setMapOutputValueClass(Text.class);
```

```
    job.setOutputKeyClass(Text.class);
```

```
    job.setOutputValueClass(Text.class);
```

```
    FileInputFormat.addInputPath(job, new Path(args[0]));
```

```
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

```
    System.exit(job.waitForCompletion(true) ? 0 : 1);
```

```
}
```

```
}
```